# Application de gestion de produits

*Rapport de développement*

**Préparé par :**
Jalal GRINI
(TDIA2)

**Encadré par :**
Prof. [Cherradi]

École Nationale des Sciences Appliquées, Al Hoceima
2025-2026

# Table des matières


**1 Introduction** **2**

**2 Description du projet** **3**

**3 Développement chronologique** **4**

3.1 Étape 1 : Conception du modèle de données . . . . . 4

3.1.1 Produit.java . . . . . 4

3.1.2 User.java . . . . . 4

3.2 Étape 2 : Couche de persistance . . . . . 5

3.2.1 ProduitDAO.java . . . . . 5

3.2.2 ProduitImpl.java . . . . . 5

3.2.3 UserDAO.java . . . . . 6

3.3 Étape 3 : Couche logique métier . . . . . 6

3.3.1 ProduitMetier.java . . . . . 6

3.3.2 ProduitMetierImpl.java . . . . . 7

3.4 Étape 4 : Authentification et gestion de session . . . . . 7

3.4.1 SessionFilter.java . . . . . 7

3.4.2 loginhandler.java . . . . . 8

3.5 Étape 5 : Contrôleurs CRUD . . . . . 8

3.5.1 AddProduitServlet.java . . . . . 8

3.5.2 ListProduitsServlet.java . . . . . 9

3.5.3 EditProduitServlet.java . . . . . 9

3.5.4 UpdateProduitServlet.java . . . . . 10

3.5.5 DeleteProduitServlet.java . . . . . 10

3.6 Étape 6 : Couche de présentation et descripteur de déploiement . . . . . 11

3.6.1 web.xml . . . . . 11

3.6.2 login.jsp . . . . . 11

3.6.3 index.jsp . . . . . 12

3.6.4 list.jsp . . . . . 12

**4 Implémentation** **13**

**5 Conclusion** **14**

# Chapitre 1

# Introduction

Ce projet est une application web Java pour gérer un catalogue de produits avec une authentification de base. Il utilise le modèle MVC avec des Servlets, JSP, une couche DAO simple en mémoire et un contrôle d'accès basé sur la session.

# Chapitre 2

# Description du projet

- **Problématique :** Construire un petit système de gestion de produits qui supporte la connexion, l'affichage, l'ajout, la modification et la suppression de produits.
- **Exigences fonctionnelles :** Authentifier les utilisateurs, protéger les pages via un filtre de session, afficher une liste de produits et permettre des opérations CRUD depuis une interface web.
- **Processus de travail :** L'implémentation a progressé de la conception du modèle de données à la création du DAO, puis à la logique métier, à l'authentification, aux contrôleurs Servlet et enfin aux vues JSP.

# Chapitre 3

# Développement chronologique

Ce chapitre présente le projet dans l'ordre exact de sa construction, avec les extraits de code essentiels des fichiers sources.

## 3.1 Étape 1 : Conception du modèle de données

La première étape a consisté à définir les structures de données de l'application.

### 3.1.1 Produit.java

L'entité produit stocke l'ID, le nom, la description et le prix.

```
project_script.py

public class Produit {
    private Long IdProduit;
    private String nom;
    private String descripton;
    private double prix;
    // constructors, getters and setters
}
```

### 3.1.2 User.java

L'entité utilisateur supporte le login, le mot de passe et le rôle.

```
project_script.py

public class User {
    private String login;
    private String password;
    private String role;
    // constructor, getters and setters
```

```
}
```

## 3.2 Étape 2 : Couche de persistance

Ensuite, les interfaces DAO et leurs implémentations ont été créées pour gérer le stockage en mémoire.

### 3.2.1 ProduitDAO.java

Le contrat DAO a été défini avec des méthodes d'ajout, suppression, récupération et mise à jour.

project_script.py

```java
public interface ProduitDAO {
    void addProduit(Produit p);
    void deleteProduit(Long id);
    Produit getProduitById(Long id);
    List<Produit> getAllProduits();
    void updateProduit(Produit p);
}
```

### 3.2.2 ProduitImpl.java

L'implémentation utilise une liste et précharge deux produits pour démonstration.

project_script.py

```java
public class ProduitImpl implements ProduitDAO {
    private List<Produit> produits = new ArrayList<>();
    public void init() {
        addProduit(new Produit("PC 1","Sony vaio 1",7000.0));
        addProduit(new Produit("PC 2","Sony vaio 2",6000.0));
    }
    @Override
    public void addProduit(Produit p) {
        p.setIdProduit((long) produits.size() + 1);
        produits.add(p);
    }
    @Override
    public List<Produit> getAllProduits() {
        return produits;
    }
    // delete, getById, update...
```

```
}
```

### 3.2.3 UserDAO.java

Un dépôt utilisateur en mémoire a été construit pour valider la connexion.

```
project_script.py

public class UserDAO {
    private static ArrayList<User> Users = new ArrayList<>();
    static {
        Users.add(new User("jalal","1234","admin"));
        Users.add(new User("test","1234","user"));
    }
    public User UserValid(String username, String password) {
        for (User a : Users) {
            if (a.getLogin().equals(username) && a.getPassword
                ().equals(password)) {
                 return a;
            }
        }
        return null;
    }
}
```

## 3.3 Étape 3 : Couche logique métier

Une couche métier singleton a été créée pour découpler les contrôleurs web du DAO.

### 3.3.1 ProduitMetier.java

L'interface de service expose les mêmes opérations CRUD que le DAO.

```
project_script.py

public interface ProduitMetier {
    void addProduit(Produit p);
    void deleteProduit(Long id);
    List<Produit> getAllProduits();
    Produit getProduitById(Long id);
    void updateProduit(Produit p);
}
```

### 3.3.2 ProduitMetierImpl.java

L'implémentation singleton initialise le DAO et redirige les appels.

```
project_script.py

public class ProduitMetierImpl implements ProduitMetier {
    private static ProduitMetierImpl instance;
    private ProduitDAO dao;
    private ProduitMetierImpl() {
        dao = new ProduitImpl();
        ((ProduitImpl) dao).init();
    }
    public static ProduitMetierImpl getInstance() {
        if (instance == null) {
            instance = new ProduitMetierImpl();
        }
        return instance;
    }
    @Override
    public List<Produit> getAllProduits() {
        return dao.getAllProduits();
    }
    // add, delete, update, getById...
}
```

## 3.4 Étape 4 : Authentification et gestion de session

L'authentification a été ajoutée avant la construction du flux de gestion des produits.

### 3.4.1 SessionFilter.java

Un filtre garantit que seuls les utilisateurs authentifiés accèdent aux pages protégées.

```
project_script.py

public class SessionFilter extends HttpFilter implements
    Filter {
    public void doFilter(ServletRequest request,
        ServletResponse response, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpSession session = req.getSession(false);
        boolean loggedIn = session != null && session.
            getAttribute("user") != null;
```

```
        if (loggedIn) {
            chain.doFilter(request, response);
        } else {
            res.sendRedirect(req.getContextPath() + "/login.jsp");
        }
    }
}
```

### 3.4.2 loginhandler.java

Le servlet de connexion valide les identifiants et redirige selon le rôle.

project_script.py

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    String login = request.getParameter("login");
    String password = request.getParameter("password");
    User u = dao.UserValid(login, password);
    if (u != null) {
        HttpSession S = request.getSession();
        S.setAttribute("user", u);
        if (u.getRole().equals("admin")) {
            request.getRequestDispatcher("index.jsp").forward(request, response);
        } else {
            request.getRequestDispatcher("list.jsp").forward(request, response);
        }
    } else {
        request.setAttribute("erreur", "login ou mot de pass est invalide");
        request.getRequestDispatcher("/login.jsp").forward(request, response);
    }
}
```

## 3.5 Étape 5 : Contrôleurs CRUD

Le cycle de vie du produit a été implémenté par des contrôleurs Servlet dédiés.

### 3.5.1 AddProduitServlet.java

Utilisé pour créer un nouveau produit à partir des données du formulaire.

```
project_script.py

protected void doPost(HttpServletRequest req,
    HttpServletResponse resp)
        throws ServletException, IOException {
    String nom = req.getParameter("nom");
    String description = req.getParameter("description");
    Double prix = Double.parseDouble(req.getParameter("prix"))
        ;
    Produit p = new Produit(nom, description, prix);
    metier.addProduit(p);
    req.setAttribute("listeProduits", metier.getAllProduits())
        ;
    req.getRequestDispatcher("index.jsp").forward(req, resp);
}
```

### 3.5.2 ListProduitsServlet.java

Affiche tous les produits ou un produit unique si un ID est fourni.

```
project_script.py

protected void doGet(HttpServletRequest req,
    HttpServletResponse resp)
        throws ServletException, IOException {
    String idParam = req.getParameter("idProduit");
    List<Produit> liste = new ArrayList<>();
    if (idParam != null && !idParam.isEmpty()) {
        Long id = Long.parseLong(idParam);
        Produit p = metier.getProduitById(id);
        if (p != null) liste.add(p);
    } else {
        liste = metier.getAllProduits();
    }
    req.setAttribute("listeProduits", liste);
    req.getRequestDispatcher("index.jsp").forward(req, resp);
}
```

### 3.5.3 EditProduitServlet.java

Charge un produit existant dans le formulaire pour modification.

```
project_script.py
```

```java
protected void doGet(HttpServletRequest req,
    HttpServletResponse resp)
        throws ServletException, IOException {
    Long id = Long.parseLong(req.getParameter("id"));
    Produit p = metier.getProduitById(id);
    req.setAttribute("produitEdit", p);
    req.setAttribute("listeProduits", metier.getAllProduits())
        ;
    req.getRequestDispatcher("index.jsp").forward(req, resp);
}
```

### 3.5.4 UpdateProduitServlet.java

Met à jour un produit à partir du formulaire de modification.

```
project_script.py
```

```java
protected void doPost(HttpServletRequest req,
    HttpServletResponse resp)
        throws ServletException, IOException {
    Long id = Long.parseLong(req.getParameter("idProduit"));
    Produit p = new Produit();
    p.setIdProduit(id);
    p.setNom(req.getParameter("nom"));
    p.setDescripton(req.getParameter("description"));
    p.setPrix(Double.parseDouble(req.getParameter("prix")));
    metier.updateProduit(p);
    req.setAttribute("listeProduits", metier.getAllProduits())
        ;
    req.getRequestDispatcher("index.jsp").forward(req, resp);
}
```

### 3.5.5 DeleteProduitServlet.java

Supprime un produit et actualise la liste.

```
project_script.py
```

```java
protected void doGet(HttpServletRequest req,
    HttpServletResponse resp)
        throws ServletException, IOException {
    Long id = Long.parseLong(req.getParameter("id"));
    metier.deleteProduit(id);
    resp.sendRedirect("listProduits");
}
```

## 3.6 Étape 6 : Couche de présentation et descripteur de déploiement

Enfin, les pages JSP et la configuration de routage ont été ajoutées.

### 3.6.1 web.xml

Le descripteur de déploiement mappe les URL aux servlets et applique le filtre d'authentification.

project_script.py

```
<filter>
    <filter-name>AuthFilter</filter-name>
    <filter-class>filters.SessionFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>AuthFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
<servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>web.loginhandler</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/loginhandler</url-pattern>
</servlet-mapping>
```

### 3.6.2 login.jsp

La page de connexion récupère les identifiants et affiche les erreurs de validation.

project_script.py

```
<form action="loginhandler" method="post">
    login: <input type="text" name="login" ><br>
    mot de pass: <input type="password" name="password" ><br>
    <input type="submit" value="login">
</form>
```

### 3.6.3 index.jsp

La page principale admin affiche le formulaire de produit et le tableau des produits.

```
project_script.py

<form action="${produitEdit != null ? 'updateProduit' : 'addProduit'}" method="post">
    <input type="hidden" name="idProduit" value="${produitEdit.idProduit}" />
    Nom: <input type="text" name="nom" value="${produitEdit.nom}" required />
    Description: <input type="text" name="description" value="${produitEdit.description}" required />
    Prix: <input type="text" name="prix" value="${produitEdit.prix}" required />
    <input type="submit" value="${produitEdit != null ? 'Modifier' : 'Ajouter'}" />
</form>
```

### 3.6.4 list.jsp

Une vue simplifiée pour les utilisateurs afin de parcourir les produits.

```
project_script.py

<table border="1" cellpadding="5">
    <tr><th>ID</th><th>Nom</th><th>Description</th><th>Prix</th></tr>
    <c:forEach var="p" items="${listeProduits}">
        <tr>
            <td>${p.idProduit}</td>
            <td>${p.nom}</td>
            <td>${p.description}</td>
            <td>${p.prix}</td>
        </tr>
    </c:forEach>
</table>
```

# Chapitre 4

# Implémentation

L'implémentation suit le flux MVC standard :

- Modèle : entités ‘Produit‘ et ‘User‘.
- DAO : ‘ProduitDAO‘ et ‘ProduitImpl‘ gèrent les données en mémoire.
- Service : ‘ProduitMetierImpl‘ centralise la logique métier produit.
- Contrôleur : les servlets traitent les requêtes et redirigent vers les pages JSP.
- Vue : les pages JSP affichent les formulaires et les tableaux en utilisant JSTL et des attributs de requête.

# Chapitre 5

# Conclusion

Ce projet démontre la construction chronologique d'une application web Java MVC avec authentification et fonctionnalités CRUD de base. L'implémentation finale est professionnelle, modulaire et prête à être étendue vers un stockage persistant ou une navigation basée sur les rôles.